PHASE 1: SILICON SELECTION, ARCHITECTURE & CLI OS

Selecting an STMicroelectronics Core, Designing the Minimum Viable Computer & Writing NovaOS v0.1 in Bare-Metal C

1. MASTER ENGINEERING ROADMAP (PHASES 1 TO 6)

SYS-PLAN

Building an independent computer with custom graphics hardware, networking, and an operating system requires strict architectural layering. We divide the NOVA-32 machine into six sequential engineering phases. Every hardware interface and software abstraction introduced in Phase 1 is explicitly designed to scale into the multi-process, graphical, networked workstation of Phases 4 through 6 without discarding foundational code.

PHASE 1: Core CPU & CLI OS [ACTIVE]
STM32H743 selection, bus/clock bring-up, linker script, startup vector table, interrupt UART ring-buffer, SysTick timer, and interactive CLI monitor shell.

PHASE 2: Board & External Memory
Custom 4-layer PCB schematic, power regulators, 32-bit FMC bus routing for 32 MB external SDRAM (`0xC0000000`), and 64 MB Quad-SPI NOR Flash.

PHASE 3: Multitasking & File System
Preemptive context switcher (`PendSV`), MPU memory protection, SDRAM slab/heap allocator, SD/QSPI block driver, and VFS/FAT32 file system.

PHASE 4: Custom Graphics Card
External FPGA/Hardware Video Co-Processor on FMC bus, dual-port VRAM framebuffer, hardware 2D sprite/text blitter, and 640×480 / 800×600 VGA/HDMI output.

PHASE 5: Internet & TCP/IP Stack
100BASE-TX RMII Ethernet PHY (`LAN8742A`), MAC DMA descriptor rings, and custom ARP, IPv4, ICMP (`ping`), UDP, DNS, and BSD-style TCP sockets.

PHASE 6: Windowing GUI & Apps
PS/2 & USB HID keyboard/mouse driver, compositing window manager, graphical terminal emulator, HTTP web client, and user-space ELF loader.

2. STMICROELECTRONICS SILICON SELECTION & TRADEOFF ANALYSIS

HW-SILICON

To serve as the central processor of a standalone computer capable of driving a dedicated external graphics card, external SDRAM, and an Ethernet controller, our STMicroelectronics chip must provide high single-thread IPC, an external parallel memory bus (Flexible Memory Controller, or FMC), and an integrated Ethernet MAC. Below is the engineering evaluation across STMicroelectronics' primary silicon families.

STMicroelectronics Part	Core & Max Clock	Internal SRAM / Flash	External Bus (FMC) & Peripherals	Architectural Verdict
STM32F407VGT6 (High-Perf M4)	Cortex-M4F 168 MHz (210 DMIPS)	192 KB SRAM 1 MB Flash	16-bit FSMC (Static RAM only, no SDRAM controller), 10/100 Ethernet MAC	**Rejected:** Lacks SDRAM support on FSMC in 100-pin package; 168 MHz bus is bottlenecked for video blitting.
STM32F767ZIT6 (Mid-Range M7)	Cortex-M7 216 MHz (462 DMIPS)	512 KB SRAM 2 MB Flash	32-bit FMC (SDRAM + SRAM), 10/100 Ethernet MAC, Quad-SPI	**Runner-Up:** Capable bus matrix, but half the clock speed and half the internal SRAM of the H7 series.
STM32H743ZIT6 (Flagship M7)	**Dual-Issue Cortex-M7 480 MHz (1027 DMIPS)**	**1,024 KB SRAM 2 MB Dual-Bank Flash**	**32-bit FMC (SDRAM + Memory-Mapped GPU Bus), 10/100 Ethernet MAC, QSPI**	**SELECTED CPU: 64-bit AXI bus, 16 KB I/D L1 Caches, 1 MB internal RAM, deterministic bare-metal boot.**
STM32MP157C (MPU Dual-A7)	Dual Cortex-A7 + M4	708 KB SRAM External	16/32-bit DDR3L controller, Gigabit Ethernet, DSI Display	**Rejected:** Requires complex multi-stage TF-A/U-Boot/PMIC bring-up and BGA routing; obscures bare-metal OS

3. STM32H743 SYSTEM BUS MATRIX & PHYSICAL MEMORY MAP

ARCH-MMAP

Unlike legacy 8-bit or 16-bit home computers with a single shared bus, the STM32H743ZIT6 is organized around a multi-layer **64-bit AXI interconnect** and three independent AHB/APB domains (D1, D2, and D3). Understanding the physical address space is mandatory before writing a single line of OS code, because instructions, stacks, kernel heaps, and hardware registers reside at fixed 32-bit hexadecimal coordinates.

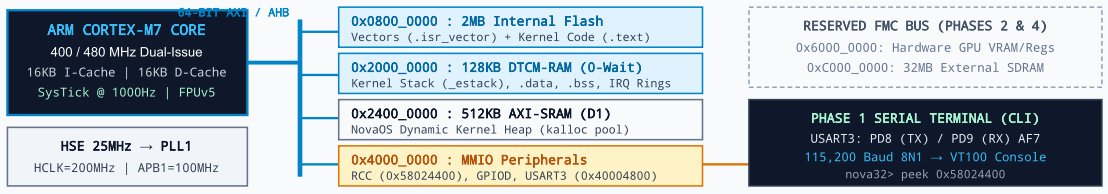


Figure 1.1 — Project NOVA-32 Physical Address Space, Clock Domain Synthesis, and Phase 1 Hardware Interconnect

4. PHASE 1 MINIMUM VIABLE HARDWARE BRING-UP CIRCUIT

HW-BRINGUP

To write, flash, and test `NovaOS v0.1` immediately while we design our multi-layer custom PCB in Phase 2, you may assemble the circuit below on an LQFP-144 breakout board or use an off-the-shelf **STMicroelectronics NUCLEO-H743ZI2** development board (which wires these exact pins out of the box). Regardless of whether you wire a bare chip or use the Nucleo carrier, the CPU executes the exact same binary instructions.

PINOUT & PHYSICAL WIRING CHECKLIST

- **Power Rails (`VDD` / `VSS`):** Connect all 11 `VDD` pins + `VDDA` to regulated **+3.3V DC** (100 nF ceramic decoupling cap within 2 mm of each pin).
- **Core Regulator (`VCAP1` , `VCAP2`):** Connect pins 71 & 106 each through a low-ESR **2.2 µF ceramic capacitor** to `GND` (internal LDO core supply).
- **Boot Config (`BOOT0` , Pin 138):** Tie via **10 kΩ pull-down resistor to GND** (`BOOT0 = 0`) so the CPU boots directly into user Flash at `0x08000000` .
- **SWD Programmer:** Connect `PA13 (SWDIO)` , `PA14 (SWCLK)` , `NRST` (with 100 nF to GND), and `GND` to an ST-LINK/V2 or V3 probe.
- **CLI Serial Console (`USART3`):** Wire `PD8 (USART3_TX, Pin 77)` and `PD9 (USART3_RX, Pin 78)` to a 3.3V USB-UART bridge (or onboard ST-LINK VCP).

CLOCK TREE & REGISTER TARGET STATE

- **Boot Default Clock:** At power-on reset, the STM32H743 runs on its internal **64 MHz HSI** oscillator (`SYSCLK = 64 MHz` , `APB1 = 64 MHz`).
- **Deterministic Phase 1 Bring-Up:** Running Phase 1 on the 64 MHz internal HSI oscillator eliminates external crystal solder-joint failures during initial OS bring-up.
- **High-Speed PLL1 Upgrade (400 MHz):** Once CLI boot is verified, `PLL1` locks onto the external oscillator (`DIVM1=5, DIVN1=160, DIVP1=2`) for 400 MHz `SYSCLK` and 200 MHz `HCLK` .
- **USART3 Baud Divisor (`BRR`):** For oversampling by 16 at 64 MHz f_{CK} and 115,200 baud: `BRR = 64,000,000 ÷ 115,200 = 556 (0x022C)` .
- **Visual Heartbeat LED:** Wire `PB0` (Green LED) via a 330 Ω resistor to GND for hardware panic/status signaling.

CRITICAL HARDWARE RULE: STM32H7 POWER SUPPLY CONFIGURATION (LDO VS. SMPS)

On the STM32H743, the `PWR_CR3` register (`0x5802480C`) controls the internal core voltage regulator. Writing mismatched SMPS/LDO bits locks the chip until power-cycled. For standard LDO routing (including bare LQFP-144 chips and Nucleo-H743ZI2 boards), always set `LDOEN = 1` (Bit 1) and wait for `PWR_CSR1.ACTVOSRDY` (Bit 13) before scaling clock frequencies.

5. BARE-METAL MEMORY LAYOUT & LINKER SCRIPT (NOVA32.LD) SW-LINKER

When you compile a program on Windows or Linux, an existing OS loads your executable into virtual memory. Because we are building the computer and OS from nothing, we must instruct the GNU linker (`arm-none-eabi-ld`) exactly where every byte resides in physical silicon. Upon hardware reset, the Cortex-M7 reads two 32-bit words from the start of Flash: the initial Main Stack Pointer (`_estack`) at `0x08000000` and the entry address of `Reset_Handler` at `0x08000004`.

FILE 1 OF 5: nova32.ld (GNU LD Bare-Metal Linker Script) **Target: STM32H743ZIT6**

```
/* Project NOVA-32 Phase 1 Linker Script — Maps Flash, DTCM-RAM (0-wait), and AXI-SRAM */
ENTRY(Reset_Handler)

MEMORY {
  FLASH   (rx)  : ORIGIN = 0x08000000, LENGTH = 2048K  /* Non-volatile Kernel & Vector ROM */
  DTCMRAM (xrw) : ORIGIN = 0x20000000, LENGTH = 128K   /* Zero-Wait-State Core Stack & Data */
  AXISRAM (xrw) : ORIGIN = 0x24000000, LENGTH = 512K   /* Primary Kernel Heap Pool (D1)     */
}

_estack = ORIGIN(DTCMRAM) + LENGTH(DTCMRAM);           /* Top of 128KB DTCM = 0x20020000    */
_heap_start = ORIGIN(AXISRAM);                         /* Start of 512KB Heap = 0x24000000  */
_heap_end   = ORIGIN(AXISRAM) + LENGTH(AXISRAM);       /* End of 512KB Heap   = 0x24080000  */

SECTIONS {
  .isr_vector : {
    . = ALIGN(4); KEEP(*(.isr_vector)) . = ALIGN(4);
  } > FLASH

  .text : {
    . = ALIGN(4); *(.text*) *(.rodata*) . = ALIGN(4);
    _etext = .;
  } > FLASH

  _sidata = LOADADDR(.data);
  .data : {
    . = ALIGN(4); _sdata = .; *(.data*) . = ALIGN(4); _edata = .;
  } > DTCMRAM AT> FLASH

  .bss : {
    . = ALIGN(4); _sbss = .; *(.bss*) *(COMMON) . = ALIGN(4); _ebss = .;
  } > DTCMRAM
}
```

6. HARDWARE VECTOR TABLE & RESET BOOTSTRAP (STARTUP.C) SW-BOOT

Before any C function with global variables can execute, our `Reset_Handler` must copy initialized variables (`.data`) from non-volatile Flash (`_sidata`) into volatile DTCM-RAM (`_sdata` to `_edata`), zero-fill uninitialized globals (`.bss`), enable the hardware Floating-Point Unit (FPU), and transfer control to `kernel_main()`. We also wire the `SysTick_Handler` (IRQ -1) and `USART3_IRQHandler` (External IRQ 39) in the vector table.

FILE 2 OF 5: startup.c (Cortex-M7 Vector Table & C Runtime Bootstrap) **Section: .isr_vector**

```
#include <stdint.h>

extern uint32_t _estack, _sidata, _sdata, _edata, _sbss, _ebss;
extern void kernel_main(void);
extern void SysTick_Handler(void);
extern void USART3_IRQHandler(void);

void Default_Handler(void) { while (1) { __asm__ volatile("nop"); } }

void Reset_Handler(void) {
    /* 1. Copy .data section from Flash LMA (_sidata) to DTCM-RAM VMA (_sdata.._edata) */
    uint32_t *src = &_sidata, *dst = &_sdata;
    while (dst < &_edata) { *dst++ = *src++; }

    /* 2. Zero-initialize the .bss section in DTCM-RAM (_sbss.._ebss) */
```

7. HARDWARE ABSTRACTION LAYER: MMIO & INTERRUPT UART DRIVER (HAL_UART.C)

SW-HAL

An operating system must never freeze the CPU waiting for a human to press a key. We therefore design a non-blocking, interrupt-driven console driver for `USART3`. Whenever a byte arrives on `PD9 (RX)`, the STM32H743 Nested Vectored Interrupt Controller (NVIC) preempts the CPU, invokes `USART3_IRQHandler`, reads `USART3_RDR`, and pushes the character into a lock-free 256-byte circular ring buffer.

> **DIRECT REGISTER ARITHMETIC: ENABLING CLOCKS, ALTERNATE FUNCTIONS & NVIC IRQ 39**
>
> To activate `USART3` on `PD8/PD9`: (1) Set Bit 3 (`GPIODEN`) in `RCC_AHB4ENR (0x580244E0)` and Bit 18 (`USART3EN`) in `RCC_APB1LENR (0x580244E8)`. (2) Set `GPIOD_MODER` bits `[19:16] = 0b1010` (Alternate Function mode) and `GPIOD_AFRH` bits `[7:0] = 0x77` (`AF7` for `PD8/PD9`). (3) Enable NVIC IRQ 39 by setting Bit `(39 - 32) = 7` in `NVIC_ISER1 (0xE000E104)`.

FILE 3 OF 5: hal_uart.c (Direct Register GPIO, RCC & Interrupt Ring-Buffer UART Driver) — **115,200 8N1 | VT100**

```c
#include <stdint.h>

/* Memory-Mapped I/O (MMIO) Physical Register Definitions for STM32H743 */
#define REG32(addr)        (*(volatile uint32_t *)(addr))
#define RCC_AHB4ENR        REG32(0x580244E0UL)
#define RCC_APB1LENR       REG32(0x580244E8UL)
#define GPIOB_MODER        REG32(0x58020400UL)
#define GPIOB_ODR          REG32(0x58020414UL)
#define GPIOD_MODER        REG32(0x58020C00UL)
#define GPIOD_AFRH         REG32(0x58020C24UL)
#define USART3_CR1         REG32(0x40004800UL)
#define USART3_BRR         REG32(0x4000480CUL)
#define USART3_ISR         REG32(0x4000481CUL)
#define USART3_ICR         REG32(0x40004820UL)
#define USART3_RDR         REG32(0x40004824UL)
#define USART3_TDR         REG32(0x40004828UL)
#define NVIC_ISER1         REG32(0xE000E104UL)

#define RX_RING_SIZE       256U
static volatile char     g_rx_ring[RX_RING_SIZE];
static volatile uint32_t g_rx_head = 0U, g_rx_tail = 0U;

void hal_uart_init(uint32_t apb1_clk_hz, uint32_t baud) {
    /* 1. Enable GPIOB (Heartbeat LED), GPIOD (UART Pins), and USART3 Peripheral Clocks */
    RCC_AHB4ENR  |= (1UL << 1) | (1UL << 3);
    RCC_APB1LENR |= (1UL << 18);
    (void)RCC_APB1LENR; /* Read-back pipeline flush */

    /* 2. Configure PB0 as Output (Status LED) and PD8/PD9 as Alternate Function AF7 */
    GPIOB_MODER = (GPIOB_MODER & ~(3UL << 0)) | (1UL << 0);
    GPIOD_MODER = (GPIOD_MODER & ~(0xFUL << 16)) | (0xAUL << 16);
    GPIOD_AFRH  = (GPIOD_AFRH  & ~(0xFFUL << 0)) | (0x77UL << 0);

    /* 3. Configure USART3: Disable UE, set BRR, enable RXNEIE (Bit 5), TE (Bit 3), RE (Bit 2),
UE (Bit 0) */
    USART3_CR1 = 0U;
    USART3_BRR = (apb1_clk_hz + (baud / 2U)) / baud;
    USART3_CR1 = (1UL << 5) | (1UL << 3) | (1UL << 2) | (1UL << 0);

    /* 4. Unmask USART3 Global Interrupt (IRQ 39 -> ISER1 Bit 7) in Cortex-M7 NVIC */
    NVIC_ISER1 = (1UL << (39U - 32U));
}

void USART3_IRQHandler(void) {
    if (USART3_ISR & (1UL << 5)) { /* RXNE: Receive Data Register Not Empty */
        char c = (char)(USART3_RDR & 0xFFU);
        uint32_t next = (g_rx_head + 1U) & (RX_RING_SIZE - 1U);
        if (next != g_rx_tail) { g_rx_ring[g_rx_head] = c; g_rx_head = next; }
    }
    USART3_ICR = 0x0FUL; /* Clear any framing/overrun error flags */
```

8. NOVAOS V0.1 KERNEL: SYSTICK, HEAP ALLOCATOR & CLI PARSER (KERNEL.C — PART 1) OS-CORE

With our hardware bootstrap and interrupt-driven serial driver in place, we now build the core of **NovaOS v0.1**. First, we configure the Cortex-M7 `SysTick` 24-bit hardware timer (`0xE000E010`) to fire an interrupt every 1 millisecond (`1000 Hz`), giving our OS a deterministic system clock and toggling the `PB0` heartbeat LED every 500 ms. Second, we implement a page-aligned kernel heap allocator (`kalloc`) over the 512 KB AXI-SRAM region (`0x24000000`). Third, we implement an interactive line editor and command tokenizer (`argc` / `argv`).

FILE 4A OF 5: kernel.c (Part 1 — SysTick Timer, AXI-SRAM Allocator & String/CLI Engine) NovaOS v0.1 Kernel

```c
#include <stdint.h>

extern void hal_uart_init(uint32_t clk, uint32_t baud);
extern void uart_putc(char c);
extern void uart_puts(const char *s);
extern int  uart_getc_nonblock(char *out_ch);
extern void uart_puthex32(uint32_t val);
extern uint32_t _heap_start, _heap_end;

#define SYST_CSR           (*(volatile uint32_t *)0xE000E010UL)
#define SYST_RVR           (*(volatile uint32_t *)0xE000E014UL)
#define SYST_CVR           (*(volatile uint32_t *)0xE000E018UL)
#define GPIOB_ODR          (*(volatile uint32_t *)0x58020414UL)
#define SYS_CLOCK_HZ       64000000UL

static volatile uint32_t g_systick_ms = 0U;
static uint8_t          *g_heap_ptr   = (uint8_t *)&_heap_start;

void SysTick_Handler(void) {
    g_systick_ms++;
    if ((g_systick_ms % 500U) == 0U) { GPIOB_ODR ^= (1UL << 0); } /* 1Hz Heartbeat on PB0 */
}

void *kalloc(uint32_t bytes) {
    bytes = (bytes + 7U) & ~7U; /* Enforce 8-byte AAPCS alignment in AXI-SRAM */
    if ((g_heap_ptr + bytes) > (uint8_t *)&_heap_end) return (void *)0;
    void *block = g_heap_ptr;
    g_heap_ptr += bytes;
    return block;
}

/* Minimal Bare-Metal Standard Library Helpers (No libc dependency) */
int kstrcmp(const char *a, const char *b) {
    while (*a && (*a == *b)) { a++; b++; }
    return (unsigned char)*a - (unsigned char)*b;
}

uint32_t kstrtoul_hex(const char *s) {
    if (s[0] == '0' && (s[1] == 'x' || s[1] == 'X')) s += 2;
    uint32_t val = 0U;
    while (*s) {
        char c = *s++;
        uint32_t d = (c >= '0' && c <= '9') ? (c - '0') :
                     (c >= 'a' && c <= 'f') ? (c - 'a' + 10) :
                     (c >= 'A' && c <= 'F') ? (c - 'A' + 10) : 0xFFU;
        if (d == 0xFFU) break;
        val = (val << 4) | d;
    }
    return val;
}

void uart_putdec(uint32_t n) {
    char buf[12]; int i = 0;
    if (n == 0U) { uart_putc('0'); return; }
    while (n > 0U) { buf[i++] = (char)('0' + (n % 10U)); n /= 10U; }
    while (i > 0)  { uart_putc(buf[--i]); }
}
```

9. BUILT-IN OS COMMANDS, SHELL LOOP & BUILD MAKEFILE (KERNEL.C — PART 2)

OS-SHELL

Below is the second half of `kernel.c` and the complete `Makefile`. The shell provides direct hardware inspection (`peek`, `poke`, `hexdump`), dynamic memory allocation testing (`alloc`), CPU/RAM diagnostics (`sysinfo`), and soft reset (`reboot`).

FILE 4B & 5 OF 5: kernel.c (Part 2 — CLI Commands & Main Loop) + Makefile — GNU Arm Embedded Toolchain

```
static void cli_execute(char *line) {
    char *argv[8]; int argc = 0;
    while (*line && argc < 8) {
        while (*line == ' ') *line++ = '\0';
        if (!*line) break;
        argv[argc++] = line;
        while (*line && *line != ' ') line++;
    }
    if (argc == 0) return;

    if (kstrcmp(argv[0], "help") == 0) {
        uart_puts("Commands: help | sysinfo | uptime | clear | peek <addr> | poke <addr> <val> |
hexdump <addr> | alloc <bytes> | reboot\n");
    } else if (kstrcmp(argv[0], "sysinfo") == 0) {
        uart_puts("CPU: STM32H743ZIT6 Cortex-M7 | CPUID: "); uart_puthex32(*(volatile uint32_t
*)0xE000ED00UL);
        uart_puts("\nAXI-SRAM Heap Used: "); uart_putdec((uint32_t)(g_heap_ptr - (uint8_t
*)&_heap_start));
        uart_puts(" / 524288 bytes\n");
    } else if (kstrcmp(argv[0], "uptime") == 0) {
        uart_puts("Uptime: "); uart_putdec(g_systick_ms / 1000U); uart_putc('.');
uart_putdec(g_systick_ms % 1000U); uart_puts(" sec\n");
    } else if (kstrcmp(argv[0], "clear") == 0) {
        uart_puts("\033[2J\033[H");
    } else if (kstrcmp(argv[0], "peek") == 0 && argc >= 2) {
        uint32_t addr = kstrtoul_hex(argv[1]);
        uart_puthex32(addr); uart_puts(" = "); uart_puthex32(*(volatile uint32_t *)addr);
uart_putc('\n');
    } else if (kstrcmp(argv[0], "poke") == 0 && argc >= 3) {
        *(volatile uint32_t *)kstrtoul_hex(argv[1]) = kstrtoul_hex(argv[2]);
        uart_puts("OK\n");
    } else if (kstrcmp(argv[0], "hexdump") == 0 && argc >= 2) {
        uint32_t base = kstrtoul_hex(argv[1]);
        for (uint32_t i = 0; i < 16U; i += 4U) {
            uart_puthex32(base + i); uart_puts(": "); uart_puthex32(*(volatile uint32_t *)(base +
i)); uart_putc('\n');
        }
    } else if (kstrcmp(argv[0], "alloc") == 0 && argc >= 2) {
        void *p = kalloc(kstrtoul_hex(argv[1]));
        uart_puts("Allocated Block @ "); uart_puthex32((uint32_t)p); uart_putc('\n');
    } else if (kstrcmp(argv[0], "reboot") == 0) {
        *(volatile uint32_t *)0xE000ED0CUL = 0x05FA0004UL; /* SCB->AIRCR SYSRESETREQ */
    } else { uart_puts("Unknown command. Type 'help'.\n"); }
}

void kernel_main(void) {
    SYST_RVR = (SYS_CLOCK_HZ / 1000UL) - 1UL; /* 1ms OS Tick */
    SYST_CVR = 0UL; SYST_CSR = 0x07UL;        /* CLKSOURCE=CPU, TICKINT=1, ENABLE=1 */
    hal_uart_init(SYS_CLOCK_HZ, 115200UL);
    uart_puts("\033[2J\033[H=== PROJECT NOVA-32 | NovaOS v0.1 (Bare-Metal CLI Kernel)
===\nnova32> ");

    char cmd_buf[80]; uint32_t pos = 0U; char c;
    while (1) {
        if (!uart_getc_nonblock(&c)) { __asm__ volatile("wfi"); continue; } /* Sleep until IRQ */
        if (c == '\r' || c == '\n') {
            uart_puts("\n"); cmd_buf[pos] = '\0'; cli_execute(cmd_buf); pos = 0U;
uart_puts("nova32> ");
        } else if ((c == 0x08 || c == 0x7F) && pos > 0U) {
            pos--; uart_puts("\b \b");
```